

Physics-Informed Neural Networks

Notes for 03_pinn.ipynb: a damped pendulum

Machine Learning for Particle Physics: A Practical Guide — extra material
JCP HEP School, Chungnam National University, September 2026

These notes accompany the notebook `03_pinn.ipynb`. Everything here uses the notebook’s notation and numbers. Margin tags point to the notebook section [\[notebook: §\]](#) and to the lecture slide [\[lecture: p.\]](#) that a passage builds on. This topic was not covered in the lecture.

1 The problem

[\[notebook: §1\]](#)

A pendulum with friction, for small angles, obeys

$$\theta''(t) + 2\beta\theta'(t) + \omega_0^2\theta(t) = 0, \quad \theta(t_0) = \theta_0, \quad \theta'(t_0) = v_0. \quad (1)$$

$\theta(t)$ is the angle, β the damping coefficient, ω_0 the natural angular frequency. This is the damped harmonic oscillator; if you have seen it as $m\ddot{x} + c\dot{x} + kx = 0$, divide by m and set $2\beta = c/m$, $\omega_0^2 = k/m$.

The notebook uses

$$\beta = 0.5, \quad \omega_0 = 3, \quad t_0 = 0, \quad \theta_0 = 1, \quad v_0 = 0, \quad t \in [0, 5].$$

We want a function $\theta(t)$ that satisfies (1). A **physics-informed neural network** (PINN) is a neural network trained to be that function. Instead of learning only from measured pairs (t_i, θ_i) , it also learns from the equation itself.

2 Exact solution

[\[notebook: §2\]](#)

For $\beta < \omega_0$ (under-damped) the solution is

$$\theta(t) = e^{-\beta(t-t_0)}(A \cos \omega_d(t-t_0) + B \sin \omega_d(t-t_0)), \quad \omega_d = \sqrt{\omega_0^2 - \beta^2}, \quad (2)$$

with $A = \theta_0$ and $B = (v_0 + \beta\theta_0)/\omega_d$. We use it only to check the network at the end. In a real problem you would not have it.

3 The network

[\[notebook: §3\]](#) [\[lecture: p. 18, 20, 24\]](#)

The network is an MLP that takes one number in and gives one number out:

$$t \longrightarrow \text{MLP} \longrightarrow \hat{\theta}(t),$$

with `tanh` activation functions in the hidden layers. The hat means “the network’s estimate”, as $\hat{y}$ did in the lecture.

Two points about the activation. First, the network only outputs $\hat{\theta}(t)$; it does *not* output derivatives. We will differentiate the network to get $\hat{\theta}'(t)$ and $\hat{\theta}''(t)$, so the network must be a smooth function of t . Second, that rules out ReLU: its second derivative is zero everywhere, so the equation residual in §5 would carry no information. `tanh` is smooth, which is why the notebook uses it here even though the lecture recommended ReLU by default.

4 Derivatives from automatic differentiation

[notebook: §4] [lecture: p. 37–39]

In the lecture, backpropagation gave $\partial\mathcal{L}/\partial\theta_{\text{params}}$: the derivative of the loss with respect to the *parameters*. Here we need something different: the derivative of the network’s *output* with respect to its *input* t . Autograd does both. The same chain rule runs through the same graph; only the variable we differentiate with respect to changes.

```
theta = model(t_f)          # t_f was created with requires_grad=True

d_theta, = torch.autograd.grad(theta, t_f,
                                grad_outputs=torch.ones_like(theta), create_graph=True)
d2_theta, = torch.autograd.grad(d_theta, t_f,
                                grad_outputs=torch.ones_like(d_theta), create_graph=True)
```

`create_graph=True` matters. The derivative $\hat{\theta}'(t)$ is itself a function of the parameters, and it will go into the loss. To train through it we must be able to differentiate it again, so PyTorch has to keep the graph that produced it. Without this flag the physics loss would be a constant as far as the optimizer is concerned.

5 Loss function

[notebook: §4] [lecture: p. 33, 50–51]

Three terms. The first is the ordinary data fit from the lecture. The other two are the physics.

$$\mathcal{L}_{\text{data}} = \frac{1}{N} \sum_{i=1}^N (\hat{\theta}(t_i) - \theta_i)^2 \quad \text{measured points} \quad (3)$$

$$\mathcal{L}_{\text{eom}} = \frac{1}{N_f} \sum_{j=1}^{N_f} \left(\hat{\theta}''(t_j) + 2\beta \hat{\theta}'(t_j) + \omega_0^2 \hat{\theta}(t_j) \right)^2 \quad \text{equation of motion} \quad (4)$$

$$\mathcal{L}_{\text{init}} = (\hat{\theta}(t_0) - \theta_0)^2 + (\hat{\theta}'(t_0) - v_0)^2 \quad \text{initial conditions} \quad (5)$$

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \mathcal{L}_{\text{eom}} + \mathcal{L}_{\text{init}} \quad (6)$$

The quantity inside $\mathcal{L}_{\text{eom}}$ is the **residual**: what is left over when you plug $\hat{\theta}$ into the left-hand side of (1). For the true solution it is exactly zero at every t . Training pushes it toward zero at the points t_j .

Weights. In general one writes $\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda_f \mathcal{L}_{\text{eom}} + \lambda_{\text{IC}} \mathcal{L}_{\text{init}}$ and tunes the λ ’s. The notebook sets both to 1. They are hyperparameters in the lecture’s sense: not learned, chosen by hand, and worth changing to see what happens.

Collocation points. [notebook: §4] The t_j in $\mathcal{L}_{\text{eom}}$ are called collocation points. At these points we need *no* measurement — only the coordinate t_j . The network predicts $\hat{\theta}(t_j)$, autograd gives the derivatives, and the equation supplies the target (zero). The notebook uses $N_f = 200$ points evenly spaced on $[0, 5]$, while the measured data live on $[0, 2]$ only. So the equation provides a training signal in $[2, 5]$, where there is no data at all. This is the main advantage of a PINN.

Where the equation is allowed. The physics domain can be larger than the data domain, $\mathcal{D}_{\text{data}} \subseteq \mathcal{D}_{\text{eom}}$, but only where the same equation with the same parameters holds. Extending $[0, 5]$ to $[0, 50]$ is fine for this pendulum; extending past the point where the small-angle approximation fails, or where β changes, is not.

6 Training: data only vs. data + physics

[notebook: §5] [lecture: p. 43, 47, 52]

The data are ten noisy measurements:

$$\theta_i = \theta(t_i) + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, 0.15^2), \quad t_i \in [0, 2].$$

The noise is added to the measurements only. Collocation points carry no noise; they are just coordinates.

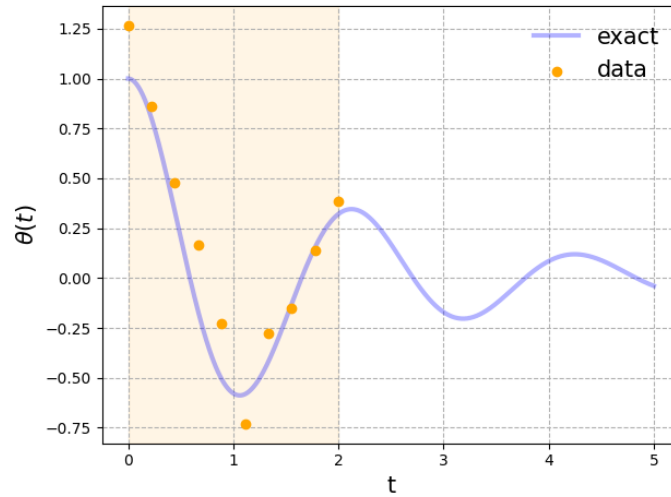


Figure 1: The exact solution and the ten noisy measurements. The shaded band is the data domain $[0, 2]$; the equation will be enforced on all of $[0, 5]$.

The notebook trains the same architecture twice from the same random seed:

	loss	sees
data only	$\mathcal{L}_{\text{data}}$	10 points on $[0, 2]$
data + physics	$\mathcal{L}_{\text{data}} + \mathcal{L}_{\text{eom}} + \mathcal{L}_{\text{init}}$	the same 10 points, plus the equation on $[0, 5]$

Both use Adam with learning rate 10^{-3} for 10 000 steps. Every step is the five-line loop from the lecture; the only change is one extra term added to `loss` before `loss.backward()`.

7 Result

[notebook: §6] [lecture: p. 50–51, 59]

Compare four curves (Fig. 2): the exact solution, the ten data points, the data-only network, and the PINN.

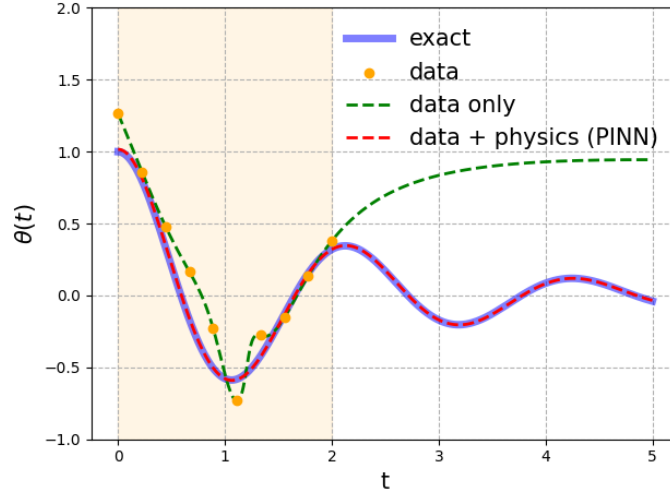


Figure 2: Same data, same network, two losses. Inside the data band both curves follow the points; outside it only the network that knows the equation keeps oscillating and decaying.

- **Inside** $[0, 2]$ both networks pass near the data. The data-only network passes *closer* to the noisy points — it is fitting the noise. The PINN does not, and should not.
- **Outside** $[0, 2]$ the data-only network has no information and does whatever the tanh units happen to do. The PINN keeps oscillating and decaying, because the equation holds there too.
- The loss curves (Fig. 3) are not comparable at face value: the PINN's loss includes two extra terms, so it sits higher. Judge the result by the plot against the exact solution, not by the final loss value.

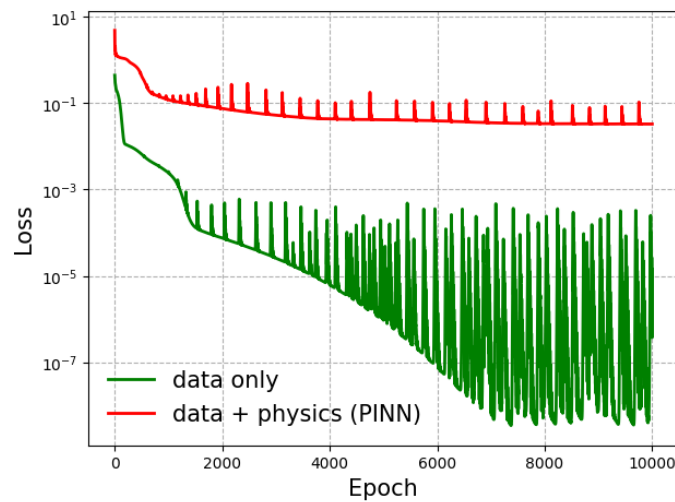


Figure 3: Training loss for the two runs. The data-only loss falls to 10^{-7} : the network has memorised ten noisy numbers. The PINN loss levels off around 10^{-1} and cannot go much lower — the data term alone is bounded below by the noise variance, $0.15^2 \approx 0.02$, and the equation term resists fitting the noise. A lower loss here is not a better model.

This is the lecture’s overfitting picture seen from a new angle. There, the fix was to penalize large weights or switch units off. Here the fix is to penalize disagreement with a physical law. The physics loss is a regularizer — one that knows what the answer should look like. It is also an inductive bias in the sense of the CNN slides: we build what we know into the training instead of hoping the data will teach it.

8 Main takeaway

The network is not a numerical ODE solver. It is a smooth function $\hat{\theta}(t)$ with adjustable parameters, and we adjust them until the function agrees with the measurements *and* satisfies the equation of motion. Same building blocks as the rest of the course — a network, a loss, autograd, a training loop — pointed at a differential equation.

Reference

- M. Raissi, P. Perdikaris, G. E. Karniadakis, *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, J. Comput. Phys. **378** (2019) 686. Preprint: [arXiv:1711.10561](https://arxiv.org/abs/1711.10561).